Report #2

Task A – File Redirection (>)

- Redirect both stdout and stderr

Task B – Pipes (|)

- Connect the output of one command to the input of another.

Studied system calls: dup2, pipe, open, close.

File Redirection code

```
for(i = 0; tokens[i] != NULL; i++){
        if(strcmp(tokens[i],">") == 0){
    »           redir_index = i;
    »           break;
        }
}
if(redir_index != -1){
        // Redirection
        if(tokens[redir_index + 1] == NULL || tokens[redir_index + 2] != NULL){
    »           // No file after '>' or extra tokens after filename error
    »       write(STDERR_FILENO,"An error has occured\n",23);
    »       //Free and continue
    »       for(i = 0; tokens[i] != NULL; i++){
    »         free(tokens[i]);
  » »       }
  » »       free(tokens);
    »       continue;
        }
}
//Terminates command before '>'
// Now tokens[0 ... redir_index-1] is the command
filename = tokens[redir_index + 1];
tokens[redir_index] = NULL;
```

```
The Only Video You Need...   Wed Feb 18  1:30:55 PM
~/Desktop/OS
iron@Ubuntu:~/Desktop/OS$ gcc minershell.c
minershell.c: In function ‘main’:
minershell.c:111:17: warning: ‘write’ reading 23 bytes from a region of size 22 [-Wstringop-overread]
  111 |                 write(STDERR_FILENO,"An error has occured\n",23);
      |                 ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
In file included from minershell.c:6:
/usr/include/unistd.h:378:16: note: in a call to function ‘write’ declared with attribute ‘access (read_only, 2, 3)’
  378 | extern ssize_t write (int __fd, const void *__buf, size_t __n) __wur
      |                ^~~~~
iron@Ubuntu:~/Desktop/OS$
```

Error after compiling code.

```
iron@Ubuntu:~/Desktop/OS$ ./a.out
minersh$ ls
a
minersh$ ls
minersh$ cd
minersh$ s
minersh$ cd ..
cd: missing argument
minersh$ ls
minersh$ cd..
minersh$ ls
minersh$ cd Desktop
cd: missing argument
minersh$ cd
cd: missing argument
minersh$ ls
minersh$ cd
cd: missing argument
minersh$ cd
cd: missing argument
minersh$
```

After debugging and compiling previous command stop working

```
  while (1) {
    /* BEGIN: TAKING INPUT */
    printf("minersh$ ");
    if(fgets(line,sizeof(line), stdin) == NULL){
»           //EOF - exit shell
»           printf("\n");
»           break;
    }
    // Remove trailing newline (fgets includes it)
    line[strcspn(line,"\n")] = '\0';

    /* END: TAKING INPUT */
    tokens = tokenize(line);
```

I modified this part of the code to use fgets and missed that had to remove the new line.

```
iron@Ubuntu:~/Desktop/OS$ gcc minershell.c
iron@Ubuntu:~/Desktop/OS$ ./a.out
minersh$ a
a
Segmentation fault (core dumped)
iron@Ubuntu:~/Desktop/OS$ ./a.out
minersh$ cd
cd
Segmentation fault (core dumped)
```

Start getting segmentation fault

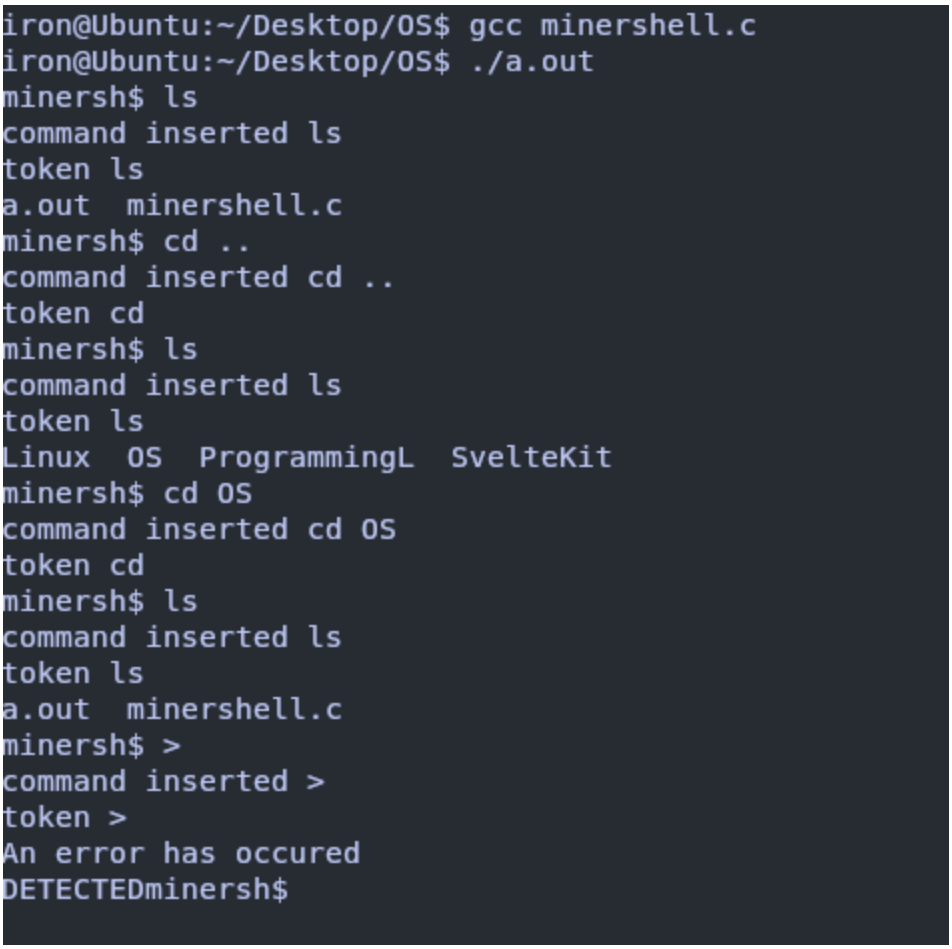
```
iron@Ubuntu:~/Desktop/OS$ gcc minershell.c
iron@Ubuntu:~/Desktop/OS$ ./a.out
minersh$ ls
command insertedls
token (null)
minersh$
```

After debugging the code that has to be modify is the tokenize to use

```
    }
    if(tokenIndex != 0){
            token[tokenIndex] = '\0';
            tokens[tokenNo] = (char *)malloc(MAX_TOKEN_SIZE * sizeof(char));
            strcpy(tokens[tokenNo++],token);
    }

    free(token);
    tokens[tokenNo] = NULL;
    return tokens;
}
```

After adding this if statement to the tokenizer



It seems to be working again and we see that we Detect the redirection > symbol

Now to implement the actual command in the child process I will take advantage of the fact that we initialize redir_index as -1 so instead of checking if execvp return -1 or not we can check for != -1 and do the write and dup2 commands

```c
if (pid == 0) {
  // Child process
  if (execvp(tokens[0], tokens) == -1) {
    // execvp failed (command not found)
    printf("Command not found: %s\n", tokens[0]);
    exit(1);
  }
} else {
  // Parent process
  int status;
  // Wait for child to finish
  waitpid(pid, &status, 0);
}
// Free allocated tokens
for(i = 0; tokens[i] != NULL; i ++)
        free(tokens[i]);
free(tokens[i]);
```

Previous code.

```c
    }
    if (pid == 0) {
      // Child process
      if (redir_index != -1){
            //Open file for writing (Create if not exist, truncate)
            int fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);
            if(fd < 0){
                  write(STDERR_FILENO,"An error has occurred\n",22);
                  exit(1);
            }
           //Redirect stdout and stderr to the file
           if(dup2(fd,STDOUT_FILENO) < 0 || dub2(fd,STDERR_FILENO) < 0){
                  write(STDERR_FILENO, "An error has occurred\n",22);
                  exit(1);
           }
           close(fd);
      }

      //Execute command
      execvp(token[0],tokens);

      // If here, execvp failed
      write(STDERR_FILENO,"An error has occurred\n",22);
      exit(1);
```

New code

```
iron@Ubuntu:~/Desktop/OS$ gcc minershell.c
iron@Ubuntu:~/Desktop/OS$ ./a.out
minersh$ ls
a.out  file.txt  minershell.c
minersh$ rm file.txt
minersh$ ls
a.out  minershell.c
minersh$ echo "Hello, world!" > out.txt
minersh$ cat out.txt
"Hello, world!"
minersh$ ls >
An error has occured
minersh$ ls -l > out.txt extra
An error has occured
minersh$ ls
a.out  minershell.c  out.txt
minersh$
```

Redirect command working and handling errors.

We now can do something similar to "|" just by checking the same errors of ">" and then handling the pipe (Task B)

```
a.out  minershell.c  out.txt
minersh$ ls | ls
SPLIT TOKENS INTO LEFT AND RIGHT COMMANDS
minersh$ ls |
An error has occurred
minersh$ |
An error has occurred
minersh$ | ls
An error has occurred
minersh$ ls | wc
SPLIT TOKENS INTO LEFT AND RIGHT COMMANDS
minersh$
```

Able to detect when to split, and when error using same logic for > command.

```c
for(i = 0; tokens[i] != NULL; i++){
        if(strcmp(tokens[i],">") == 0)redir_index = i;
        if(strcmp(tokens[i],"|") == 0)pipe_index = i;
}
// Handles PIPES
if(pipe_index != -1){
        //Validate syntax:
        if(pipe_index == 0 || tokens[pipe_index + 1] == NULL){
    »           write(STDERR_FILENO, "An error has occurred\n",22);
    »           for(i = 0; tokens[i] != NULL; i++) free(tokens[i]);
    »           free(tokens);
    »           continue;
        }
        //Split tokes into left and right commands
        printf("SPLIT TOKENS INTO LEFT AND RIGHT COMMANDS\n");

        // Free tokens and continue
        for(i=0;tokens[i] != NULL; i++) free(tokens[i]);
        free(tokens);
        continue;
}
// Handles REDIRECTION
if(redir_index != -1){
        // Redirection
        if(tokens[redir_index + 1] == NULL || tokens[redir_index + 2] != NULL){
```

Logic for errors and when to split.